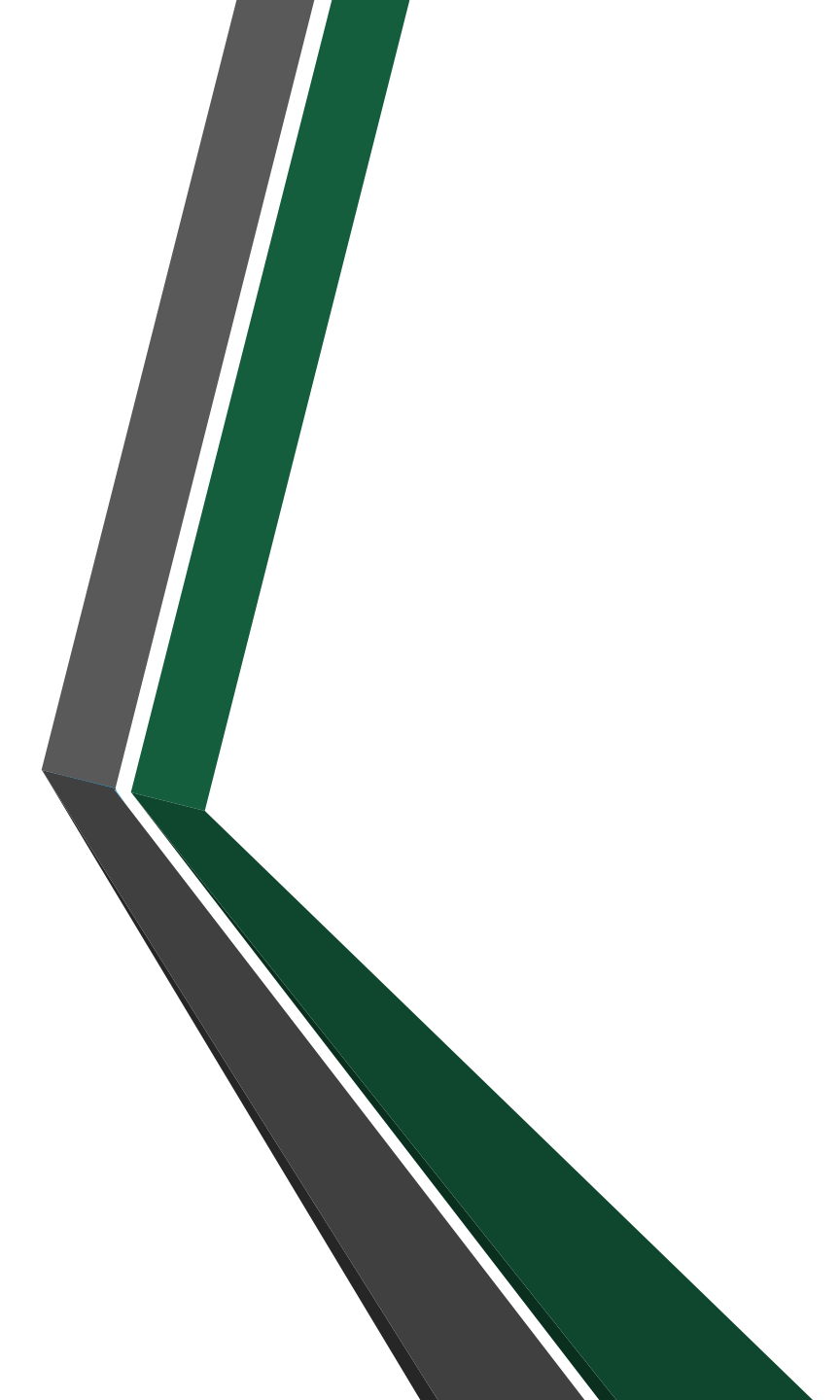




BAZE DE DATE

CURS 3

DIAGrame ENTITATE-RELAȚIE

RECAPITULARE

Algoritmul pentru proiectarea Diagramei E/R

1. identificarea **entităților** din cadrul sistemului analizat;
2. identificarea **relațiilor** (asocierilor) dintre entități și stabilirea **cardinalității**;
3. identificarea **atributelor** aferente entităților și asocierilor dintre entități;
4. stabilirea atributelor de identificare a entităților, adică stabilirea **cheilor primare**;

EXERCIȚIU RECAPITULATIV

Exercițiul 1 – Evidența școlilor de șoferi

Completați relațiile (*lucrează_la, conduce, susține, asistă, instruiește*) dintre entități și specificați cardinalitatea (maximă și minimă)

SCOALA
cod_scoala#

CLIENT
cod_client#

INSTRUCTOR
cod_instructor#

EXAMEN
cod_examen#

MASINA
cod_masina#

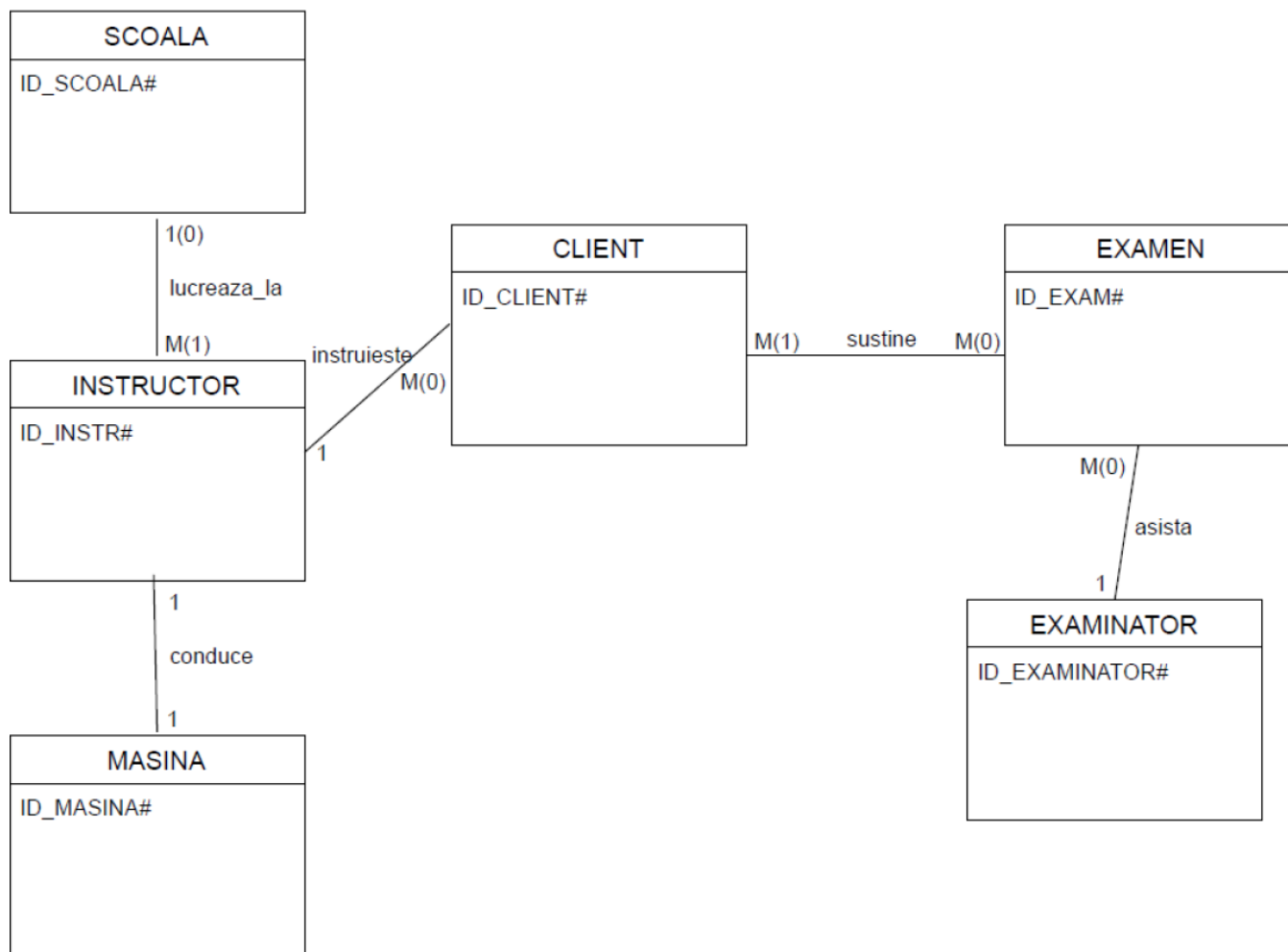
EXAMINATOR
cod_examinator#

EXERCİȚIU RECAPITULATIV

Regulile de funcționare:

- Baza de date modelează activitatea din cadrul unor școli de șoferi;
- Fiecare școală de șoferi angajează instructori care se ocupă de pregătirea clienților. În cadrul școlii pot lucra mai mulți instructori, dar un instructor nu poate lucra pentru mai multe școli de șoferi simultan. Instructorul poate face parte din cadrul unei singure școli de șoferi sau poate lucra fără a face parte, în mod obligatoriu, dintr-o școală;
- Fiecare instructor deține doar o singură mașină, aceasta aparținând unui singur instructor, și poate să instruiască mai mulți clienți. Pot exista situații în care instructorul nu are încă alocați cursanți. De asemenea, un cursant trebuie să fie pregătit de același instructor pe toată durata cursului;
- Există și situația în care un client se înscrie în cadrul școlii de șoferi doar pentru ore suplimentare de condus, ceea ce înseamnă că nu o să susțină examenul final. Cei care vor susține examenul, pot participa de mai multe ori la examen. În cadrul examenului este prezent întotdeauna un singur examinator;
- Se consideră că examenul este compus din ambele probe (SCRIS și PRACTIC). Se va proiecta doar o singură entitate – EXAMEN.

EXERCİIU RECAPITULATIV



DIAGrame ENTITATE-RELATIE

Cazuri speciale de entități, relații, attribute și modul lor de reprezentare în cadrul diagramei entitate-relație:

- Dependența (entitatea dependentă)
- Relații recursive
- Moștenirea atributelor (**subentitate** / **superentitate**)
- Generalizare, specializare
- Relații de tip 2, 3
- Excluderea relațiilor indirecte
- Eliminarea atributelor derivabile
- Relație sau atribut?
- Entitate sau relație?
- Reassignarea atributelor indirecte
- Attribute optionale

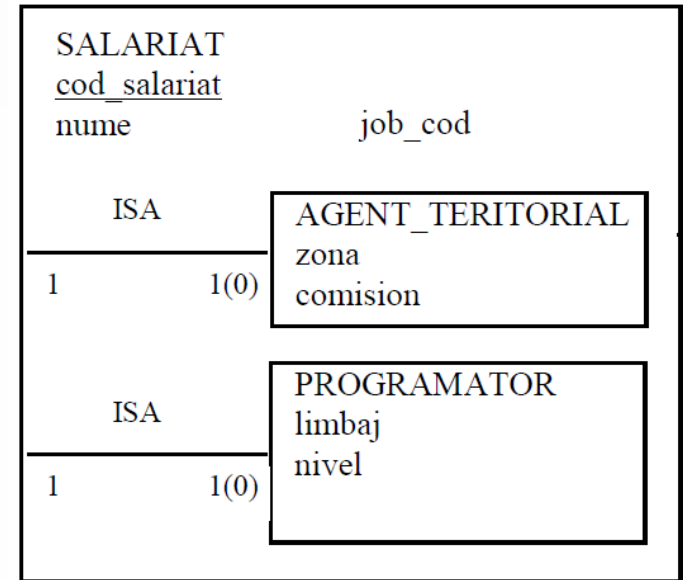
DIAGrame ENTITATE-RELAȚIE

Moștenirea atributelor

- **Subentitate (subclasă)** – submulțime a unei alte entități, numită **superentitate (superclasă)**

SALARIAT < — > PROGRAMATOR

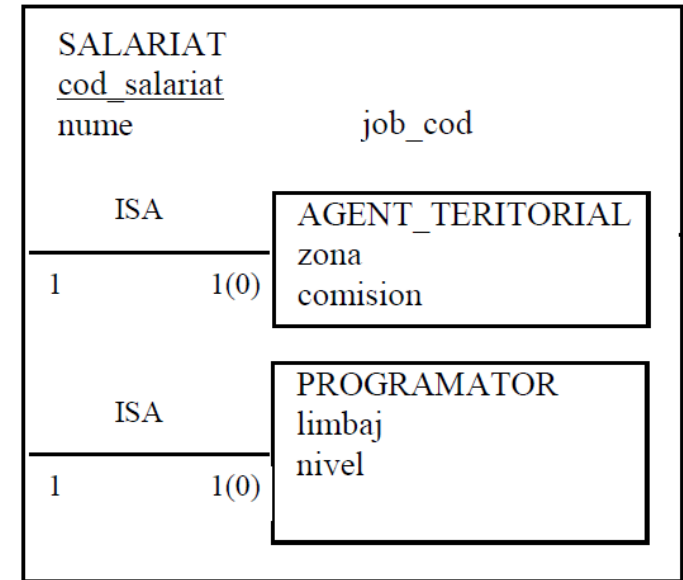
- Subentitatea se desenează prin dreptunghiuri incluse în superentitate.
- Există o relație între o subentitate și o superentitate, numită **ISA**, care are cardinalitatea maximă 1:1 și minimă 1:0.
- Cheile primare, attributele și relațiile unei superentități sunt valabile pentru orice subentitate. Afirmția reciprocă este falsă.



DIAGrame ENTITATE-RELAȚIE

Cardinalitatea dintre o **superentitate** (superclasă) și o **subentitate** (subclasă) într-o relație **ISA** este:

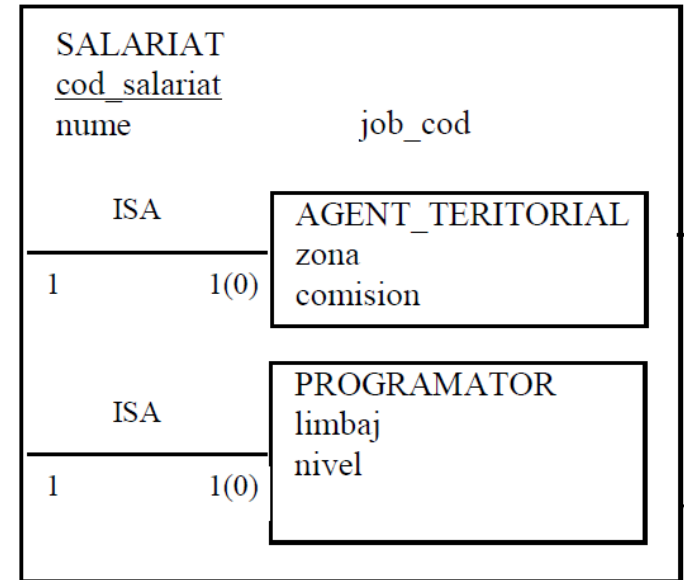
- **Maximă: 1:1** → Fiecare entitate din subclasă **corespunde exact unei** entități din superclasă.
- **Minimă: 1:0** → O entitate din superclasă poate **să nu aibă o corespondență** în subclasă.



DIAGrame ENTITATE-RELAȚIE

Aceasta înseamnă că:

- **Nu orice entitate din superclasă trebuie să aibă o reprezentare într-o subclasă, dar fiecare entitate dintr-o subclasă trebuie să fie legată de o entitate din superclasă.**
- De exemplu, într-o relație **SALARIAT → PROGRAMATOR**, nu toți salariații sunt programatori, dar orice programator **este** și un salariat.



DIAGrame ENTITATE-RELATIE

Superclasa (superentitate) / Subclasa (subentitate):

- **Superclasa** (superentitatea) este o entitate care include **subclase** (subentități) distincte, ce trebuie reprezentate în modelul de date.
- **Subclasa** are un rol distinct și, evident, este membră a unei superclase. O subclasă, fiind o entitate, poate să posede propriile subclase.

O entitate împreună cu subclasele ei, subclasele acestora și așa mai departe definește o **ierarhie de tip** (ierarhie de specializare).

DIAGrame ENTITATE-RELATIE

Cazuri speciale de entități, relații, atribute și modul lor de reprezentare în cadrul diagramei entitate-relație:

- Dependența (entitatea dependentă)
- Relații recursive
- Moștenirea atributelor (subentitate / superentitate)
- **Generalizare, specializare**
- Relații de tip 2, 3
- Excluderea relațiilor indirecte
- Eliminarea atributelor derivabile
- Relație sau atribut?
- Entitate sau relație?
- Reassignarea atributelor indirecte
- Atribute optionale

DIAGrame ENTITATE-RELATIE

Specializarea - (de la general la specific)

- Este procesul de **maximizare a diferențelor** dintre membrii unei entități, prin identificarea caracteristicilor distinctive ale acestora (**subclasa**).
- Specializarea reprezintă **procesul de împărțire** a unei superclase în **subclase mai specifice**. Cu alte cuvinte, se pornește de la un concept **general** și se **detaliază** în categorii mai restrânse.

Exemplu:

Entitatea **SALARIAT** (superclasă) este împărțită în subclase mai specifice:

- **PROGRAMATOR**
- **HR**
- **CONTABIL**

Se aplică atunci când o entitate are subtipuri specifice care trebuie modelate distinct.

DIAGrame ENTITATE-RELATIE

Generalizarea - (de la specific la general)

- Este procesul de **minimizare a diferențelor** dintre entități, prin identificarea caracteristicilor comune ale acestora. Generalizarea are ca rezultat identificarea unei **superclase** generalizate din subclasele inițiale.
- Generalizarea este **procesul invers specializării**: se identifică toate **caracteristici comune** mai multor entități, după care se **grupează** într-o superclasă.

Exemplu:

- Entitățile **PROGRAMATOR**, **HR**, **CONTABIL** au attribute comune, precum: nume, salariu, data angajării. Astfel, ele se vor **grupa** într-o superclasă comună, numită **SALARIIAT**.

Se aplică atunci când se observă că mai multe entități au caracteristici comune.

DIAGrame ENTITATE-RELATIE

Cazuri speciale de entități, relații, atribute și modul lor de reprezentare în cadrul diagramei entitate-relație:

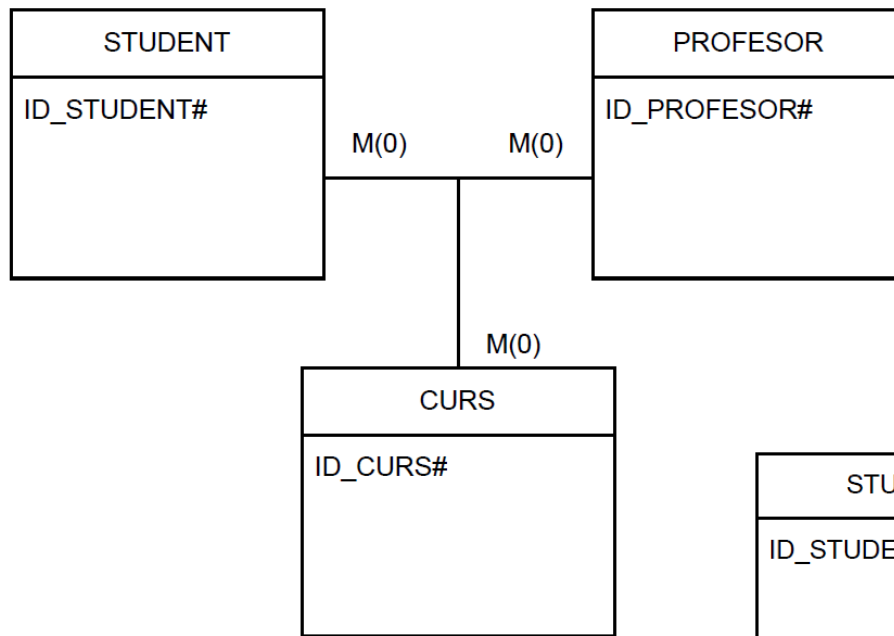
- Dependența (entitatea dependentă)
- Relații recursive
- Moștenirea atributelor (subentitate / superentitate)
- Generalizare, specializare
- **Relații de tip 2, 3**
- Excluderea relațiilor indirecte
- Eliminarea atributelor derivabile
- Relație sau atribut?
- Entitate sau relație?
- Reassignarea atributelor indirecte
- Atribute optionale

DIAGrame ENTITATE-RELAȚIE

Unele relații sunt relative la două entități și le numim de **tip 2**, iar dacă relațiile implică **mai mult de două entități**, le vom numi de **tip 3**.

OBSERVAȚIE!

- Trei relații de tip 2 sunt diferite de o relație de tip 3
- Rupând o relație de tip 3 în trei relații de tip 2, pot apărea informații incorecte



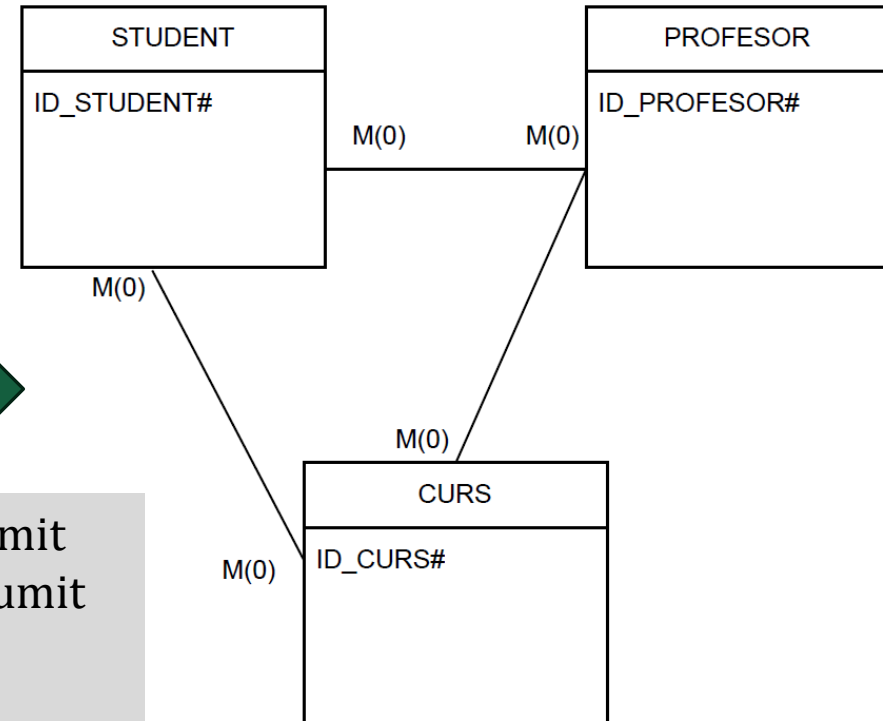
0 RELAȚIE DE TIP 3



3 RELAȚII DE TIP 2



Un student participă la un anumit curs care este predat de un anumit profesor.
 Relația exprimă faptul că **un profesor predă un anumit curs unui anumit student.**



Prin urmare, relația depinde simultan de cele trei entități: **STUDENT, CURS și PROFESOR**, ceea ce conduce la o **relație ternară**.

Relația nu descrie doar ce cursuri urmează studenții sau ce cursuri predau profesorii, ci **care profesor predă acel curs acelui student**.

În cadrul acestei relații ternare:

- Un **student** poate participa în mai multe combinații (*curs, profesor*).
- Un **profesor** poate predă mai multe combinații (*student, curs*).
- Un **curs** poate fi asociat cu mai multe combinații (*student, profesor*).
- Aceasta înseamnă că participarea fiecărei entități în relație poate avea **multiplicitate M**.

Prin urmare, relația ternară dintre **STUDENT, CURS și PROFESOR** poate avea cardinalitatea **M : M : M**.

Cardinalitatea minimă:

Cardinalitatea minimă pentru fiecare entitate poate fi **0**, deoarece participarea într-o relație poate fi opțională.

Acest lucru se justifică astfel:

- Un **student** poate să nu fie înscris la niciun curs (de exemplu, un student nou admis care încă nu și-a ales cursurile).
- Un **profesor** poate să nu predea niciun curs într-un anumit semestru (de exemplu, dacă este în concediu sau implicat exclusiv în activități de cercetare).
- Un **curs** poate să nu aibă niciun student înscris (de exemplu, un curs nou introdus în planul de învățământ).
- Prin urmare, cardinalitatea minimă pentru fiecare entitate poate fi **0**, iar cardinalitatea maximă poate fi **M**.

Cum se implementează corect într-o bază de date?

Având în vedere că relația ternară dintre **STUDENT**, **CURS** și **PROFESOR** are cardinalitatea **M : M : M**, se va implementa o **tabelă intermediară** pentru relația de tip 3.

```
CREATE TABLE INSCRIERE (  
    ID_STUDENT NUMBER(10),  
    ID_CURS NUMBER(10),  
    ID_PROFESOR NUMBER(10),  
    PRIMARY KEY (ID_STUDENT, ID_CURS, ID_PROFESOR),  
    FOREIGN KEY (ID_STUDENT) REFERENCES STUDENT(ID_STUDENT),  
    FOREIGN KEY (ID_CURS) REFERENCES CURS(ID_CURS),  
    FOREIGN KEY (ID_PROFESOR) REFERENCES PROFESOR(ID_PROFESOR)  
);
```

Această abordare:

- Evită dublările de date
- Păstrează corect relația între cele 3 entități
- Permite ca fiecare student să fie înscris la mai multe cursuri, fiecare predat de mai mulți profesori

De ce nu descompunem relația ternară în trei relații binare?

Dacă încercăm să transformăm relația ternară în trei relații binare, obținem:

STUDENT – CURS

PROFESOR – CURS

STUDENT – PROFESOR

Fiecare dintre aceste relații poate fi de tip **many-to-many**.

Problema este că această descompunere **pierde informația despre legătura dintre cele trei entități simultan**.

În relația ternară știm exact:

- ce profesor predă **un anumit curs unui anumit student**.

După descompunere, aceste informații sunt separate în trei relații diferite, iar prin combinarea lor pot apărea **asocieri incorecte**.

De exemplu, se poate deduce greșit că:

- un anumit profesor predă un curs unui student, chiar dacă acea combinație nu există în realitate.

Prin urmare, descompunerea unei relații ternare în relații binare poate conduce la **ambiguități și informații incorecte**, deoarece nu mai este păstrată dependența simultană dintre cele trei entități.

ID_STUDENT#

1

1

2

3

ID_CURS#

Baze de Date

Algoritmi

Algoritmi

Machine Learning

ID_PROFESOR#

Prof. Ionescu

Prof. Popescu

Prof. Georgescu

Prof. Ionescu

ID_CURS#

Baze de Date

Algoritmi

Machine Learning

Algoritmi

ID_STUDENT#

1

2

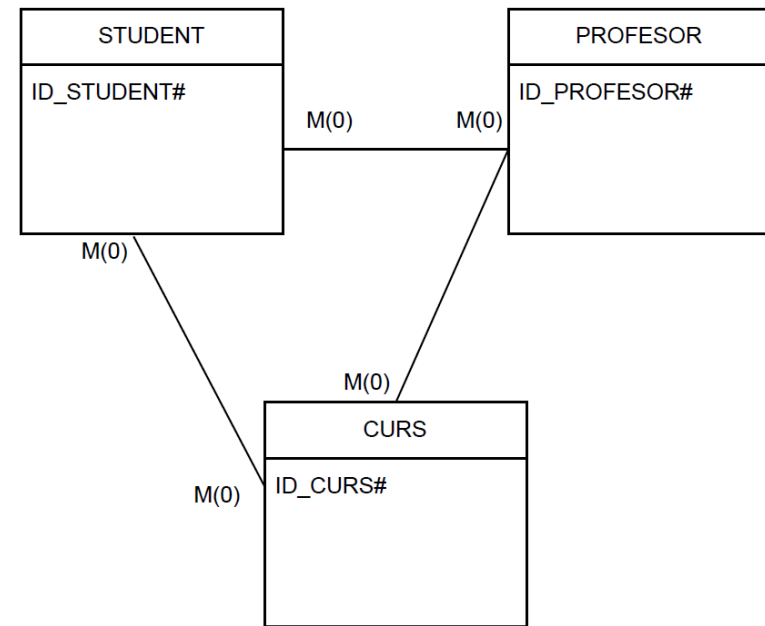
3

ID_PROFESOR#

Prof. Ionescu

Prof. Popescu

Prof. Georgescu



ID_STUDENT#

1

1

2

3

ID_CURS#

Baze de Date

Algoritmi

Algoritmi

Machine Learning

ID_PROFESOR#

Prof. Ionescu

Prof. Popescu

Prof. Georgescu

Prof. Ionescu

ID_CURS#

Baze de Date

Algoritmi

Machine Learning

Algoritmi

ID_STUDENT#

1

2

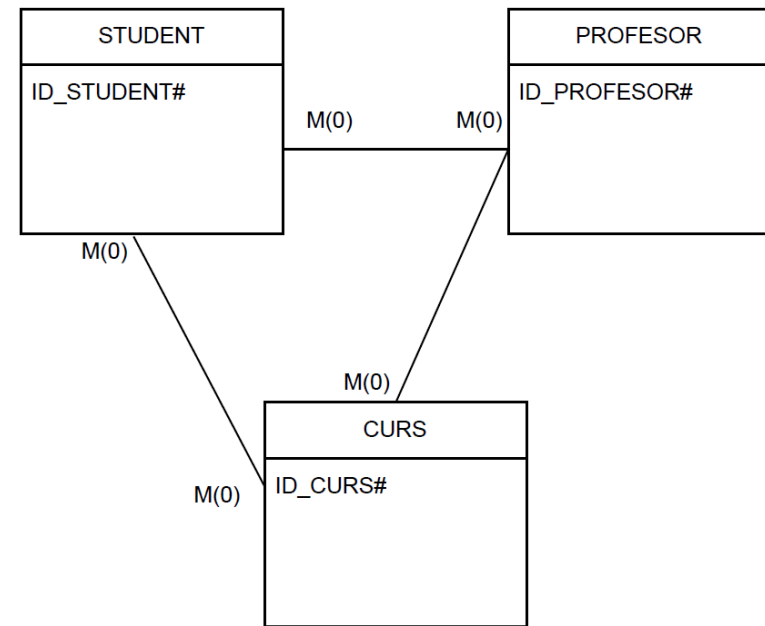
3

ID_PROFESOR#

Prof. Ionescu

Prof. Popescu

Prof. Georgescu



Avem o ambiguitate!

CONCLUZIE

O **relație ternară** este necesară atunci când relația dintre două entități depinde de o a treia entitate. Dacă această relație este descompusă în relații binare, informația despre dependența dintre cele trei entități poate fi pierdută.

EXEMPLE DE RELAȚII TERNARE CORECTE

/! Dacă relația dintre două entități depinde de o a treia, atunci este nevoie de relație ternară.

1. Un doctor prescrie un medicament unui pacient.

Pacient – Doctor – Medicament => PRESCRIPTIE(Pacient, Doctor, Medicament)

2. Un client cumpără un produs dintr-un anumit magazin.

Client – Produs – Magazin => CUMPARARE(Client, Produs, Magazin)

3. Un angajat lucrează la un proiect folosind o anumită tehnologie.

Angajat – Proiect – Tehnologie => LUCREAZA(Angajat, Proiect, Tehnologie)

4. Un student susține un examen la un profesor.

Student – Examen – Profesor => EXAMEN(Student, Profesor, Materie)

EXEMPLE DE RELAȚII TERNARE **GREȘITE**

1. Studentul participă la un curs într-o anumită sală.

STUDENT – CURS – SALĂ

De ce este greșit?

Sala nu depinde de student. Sala este asociată doar **cursului**, nu studentului. Studentul nu schimbă sala cursului.

Modelare corectă:

Două relații binare:

Student – Curs (înscriere)

Curs – Sala (se desfășoară în)

EXEMPLE DE RELAȚII TERNARE **GREȘITE**

2. CLIENT – PRODUS – PREȚ

Relație ternară greșită:

CUMPARARE(Client, Produs, Pret)

De ce este greșit?

Prețul nu este o entitate independentă, ci **un atribut al produsului sau al tranzacției.**

Modelare corectă:

Client – Produs, iar **pretul este atribut al relației:**

CUMPARARE(Client, Produs, pret)

EXEMPLE DE RELAȚII TERNARE **GREȘITE**

3. STUDENT – PROFESOR – UNIVERSITATE

Relație ternară greșită:

RELATIE(Student, Profesor, Universitate)

De ce este greșit?

Studentul și profesorul aparțin universității, independent unul de celălalt. Universitatea nu depinde de relația dintre student și profesor.

Modelare corectă:

Două relații binare:

Student – Universitate

Profesor – Universitate

UN ALT EXEMPLU DE RELAȚIE TERNARĂ

OSERVAȚIE!

Relațiile ternare **NU** au mereu cardinalitate many-to-many.

După cum spuneam, o relație ternară este o relație care implică cel puțin 3 entități simultan.

Reguli de funcționare:

- un **medic poate consulta mai mulți pacienți** – deci între MEDIC și PACIENT apare cardinalitatea many;
- un **pacient poate fi consultat de un medic într-un anumit spital**;
- un **medic poate lucra în mai multe spitale**.

EXEMPLU DE RELAȚIE TERNARĂ

Reguli de funcționare:

- un **medic** poate consulta mai mulți pacienți – deci între MEDIC și PACIENT apare cardinalitatea many;
- un **pacient** poate fi consultat de un medic într-un anumit spital;
- un **medic** poate lucra în mai multe spitale.

Se stabilesc cardinalitățile ternare:

- pentru o pereche (**PACIENT, SPITAL**) există un singur **MEDIC**;
- pentru o pereche (**MEDIC, SPITAL**) pot exista mai mulți **PACIENȚI**;
- pentru o pereche (**MEDIC, PACIENT**) pot exista mai multe **SPITALE**;

Deci relația ternară este: **1, N, N** (în ordinea: MEDIC, PACIENT, SPITAL).

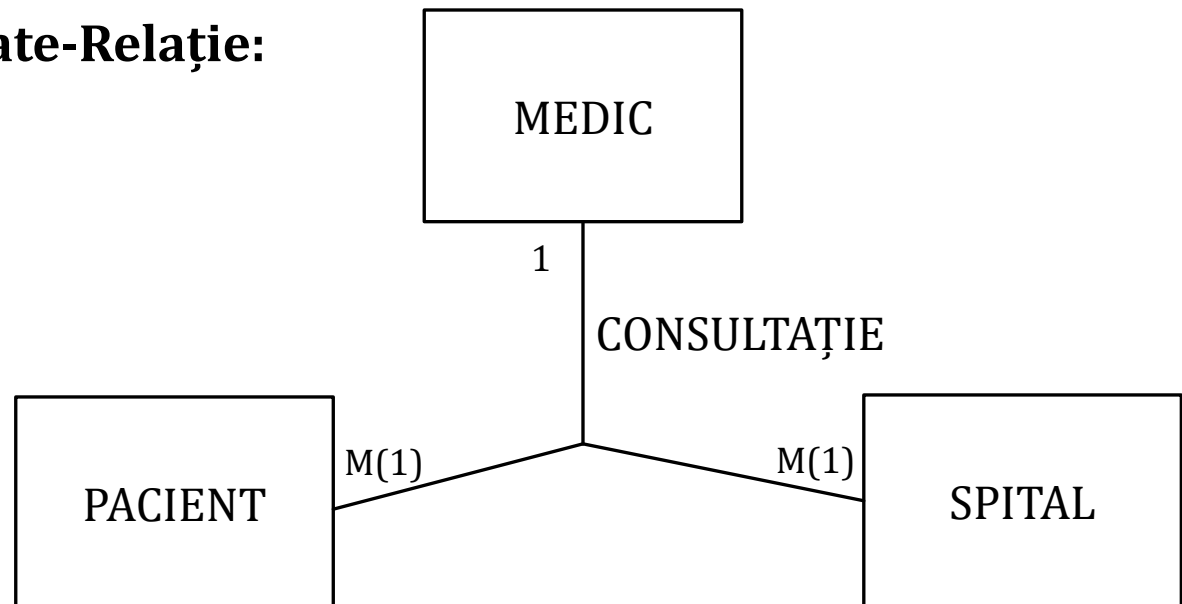
EXEMPLU DE RELAȚIE TERNARĂ

Se stabilesc cardinalitățile ternare:

- pentru o pereche (**PACIENT, SPITAL**) există **un singur MEDIC**;
- pentru o pereche (**MEDIC, SPITAL**) pot exista **mai mulți PACIENȚI**;
- pentru o pereche (**MEDIC, PACIENT**) pot exista **mai multe SPITALE**;

Deci relația ternară este: **1, N, N** (în ordinea: MEDIC, PACIENT, SPITAL).

Diagrama Entitate-Relație:



EXEMPLU DE RELAȚIE TERNARĂ

Proiectare conceptuală (schema relațională):

Entități:

MEDIC(id_medic PK, nume)

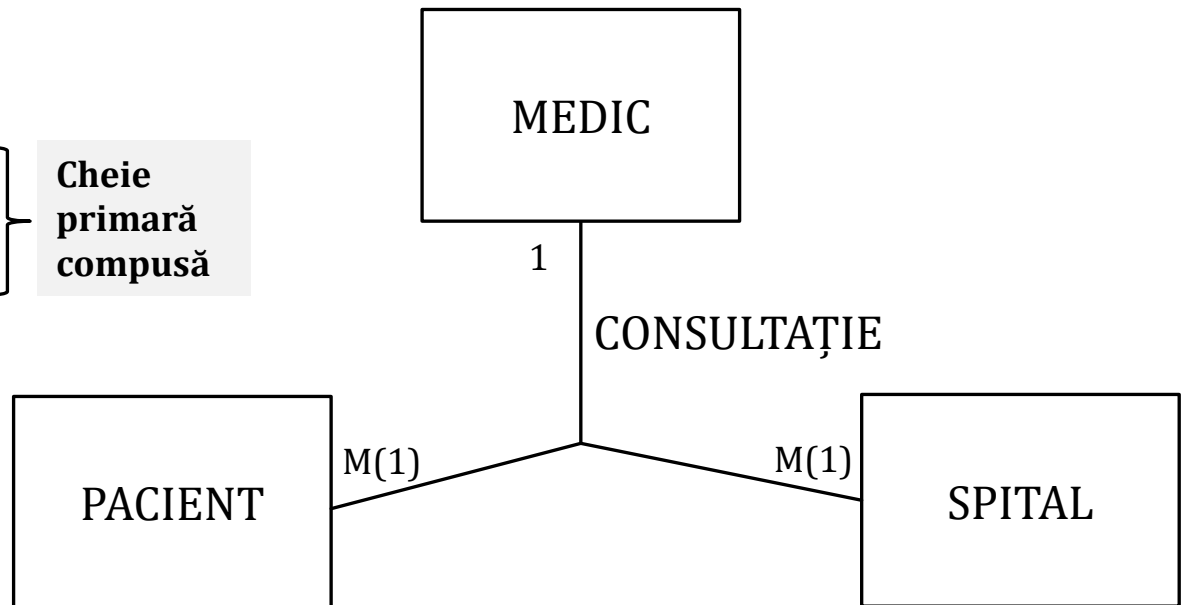
PACIENT(id_pacient PK, nume)

SPITAL(id_spital PK, nume)

Relația ternară:

CONSULTATIE

(
id_medic FK, } **Cheie**
id_pacient FK, } **primară**
id_spital FK, } **compusă**
data
)



EXEMPLU DE RELAȚIE TERNARĂ

Proiectare conceptuală (schema relațională):

Entități:

MEDIC(id_medic PK, nume)

PACIENT(id_pacient PK, nume)

SPITAL(id_spital PK, nume)

Ce constrangere este necesară pentru a respecta regulile de funcționare și a ține cont de faptul că avem cardinalitatea 1 la MEDIC?

Relația ternară:

CONSULTATIE

(

id_medic FK,

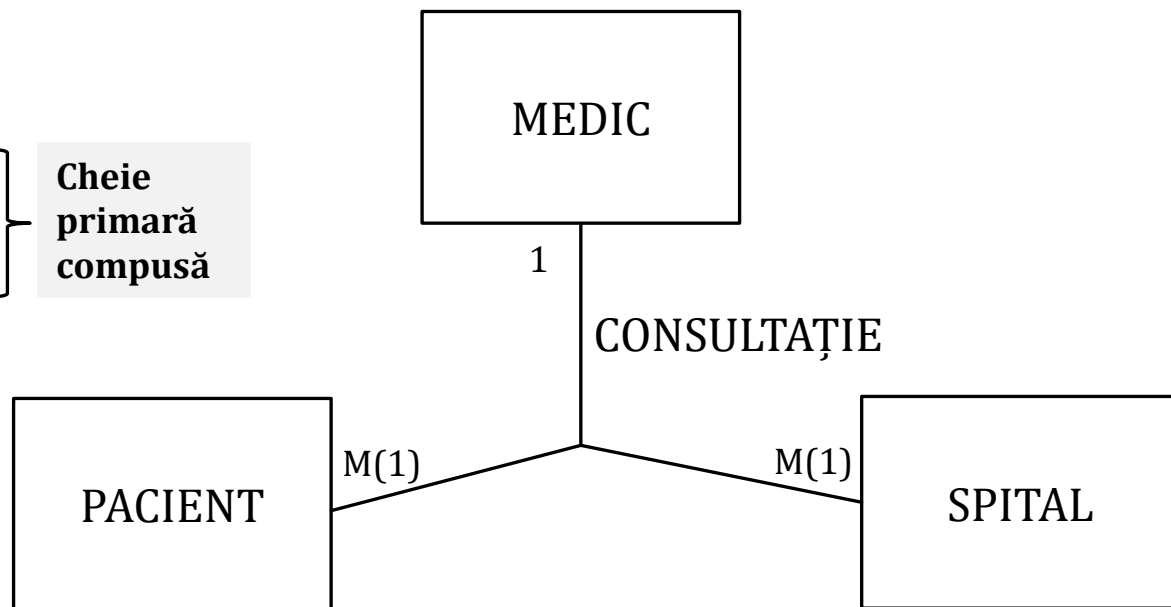
id_pacient FK,

id_spital FK,

data

)

Cheie
primară
compusă



EXEMPLU DE RELAȚIE TERNARĂ

Ce constrangere este necesară pentru a respecta regulile de funcționare și a ține cont de faptul că avem cardinalitatea 1 la MEDIC?

Pentru aceeași pereche (id_pacient, id_spital) nu pot exista doi medici

Deci:

UNIQUE(id_pacient, id_spital)

NU punem:

UNIQUE(id_medic, id_spital) – pentru că medicul trebuie să aibă mai mulți pacienți.

NU punem:

UNIQUE(id_medic, id_pacient) – pentru că medicii pot lucra în mai multe spitale.

IMPLEMENTARE

```
CREATE TABLE medic (  
    id_medic NUMBER(5) PRIMARY KEY,  
    nume VARCHAR2(100)  
);  
  
CREATE TABLE pacient (  
    id_pacient NUMBER(5) PRIMARY KEY,  
    nume VARCHAR2(100)  
);  
  
CREATE TABLE spital (  
    id_spital NUMBER(5) PRIMARY KEY,  
    nume VARCHAR2(100)  
);  
  
CREATE TABLE consultatie (  
    id_medic NUMBER(5),  
    id_pacient NUMBER(5),  
    id_spital NUMBER(5),  
    data_consult DATE,
```

```
CONSTRAINT pk_consultatie  
    PRIMARY KEY (id_medic, id_pacient,  
id_spital),  
  
CONSTRAINT fk_cons_medic  
    FOREIGN KEY (id_medic)  
REFERENCES medic(id_medic),  
  
CONSTRAINT fk_cons_pacient  
    FOREIGN KEY (id_pacient)  
REFERENCES pacient(id_pacient),  
  
CONSTRAINT fk_cons_spital  
    FOREIGN KEY (id_spital)  
REFERENCES spital(id_spital),  
  
CONSTRAINT uq_pacient_spital  
    UNIQUE (id_pacient, id_spital)  
);
```

DIAGrame ENTITATE-RELATIE

Cazuri speciale de entități, relații, atribute și modul lor de reprezentare în cadrul diagramei entitate-relație:

- Dependența (entitatea dependentă)
- Relații recursive
- Moștenirea atributelor (subentitate / superentitate)
- Generalizare, specializare
- Relații de tip 2, 3
- Excluderea relațiilor indirecte
- Eliminarea atributelor derivabile
- Relație sau atribut?
- Entitate sau relație?
- Reassignarea atributelor indirecte
- Atribute optionale

DIAGrame ENTITATE-RELAȚIE

- Trebuie **exclude** din model **relațiile indirecte** (ex: un angajat este coleg de departament cu un alt angajat) deoarece ele pot conduce la redundanță în baza de date
- **Atributele derivabile** trebuie eliminate și introduse expresii prin care aceste atribute pot fi calculate
- **EXAMPLE?**

DIAGrame ENTITATE-RELAȚIE

- Trebuie **exclude** din model **relațiile indirecte** (ex: un angajat este coleg de departament cu un alt angajat) deoarece ele pot conduce la redundanță în baza de date
- **Atributele derivabile** trebuie eliminate și introduse expresii prin care aceste atribute pot fi calculate
- **EXEMPLE?**
 - **Vârsta** -> se calculează din data nașterii
 - **Media generală** -> se calculează din notele obținute la examene
 - **Număr de studenți înscriși la un curs** -> Se calculează prin numărarea studenților asociați cursului
 - **Vechime** -> se calculează din data angajării

De ce nu salvăm atributele derivabile în baza de date?

Se pot recalcula oricând → nu se va ocupa spațiul de stocare cu informații redundante.

Se reduce riscul de inconsistență → dacă se schimbă datele de bază (ex: data nașterii), atunci vârsta se va recalcula corect.

Performanță optimizată → se evită actualizările manuale asupra unor date care pot fi generate dinamic.

De ce nu salvăm atributele derivabile în baza de date?

Se pot recalcula oricând → nu se va ocupa spațiul de stocare cu informații redundante.

Se reduce riscul de inconsistență → dacă se schimbă datele de bază (ex: data nașterii), atunci vârsta se va recalcula corect.

Performanță optimizată → se evită actualizările manuale asupra unor date care pot fi generate dinamic.

Când ar trebui totuși să stocăm un atribut derivabil?

Dacă **este folosit foarte des** și calcularea lui **ar încetini performanța** (ex: total vânzări lunare într-o bază de date foarte mare).

Dacă **nu se dorește recalcularea datelor istorice**, iar un atribut poate varia în timp (ex: salariul net, care se poate schimba în funcție de taxe și deduceri).

DIAGrame ENTITATE-RELATIE

Cazuri speciale de entități, relații, atribute și modul lor de reprezentare în cadrul diagramei entitate-relație:

- Dependența (entitatea dependentă)
- Relații recursive
- Moștenirea atributelor (subentitate / superentitate)
- Generalizare, specializare
- Relații de tip 2, 3
- Excluderea relațiilor indirecte
- Eliminarea atributelor derivabile
- **Relație sau atribut?**
- **Entitate sau relație?**
- Reassignarea atributelor indirecte
- Atribute optionale

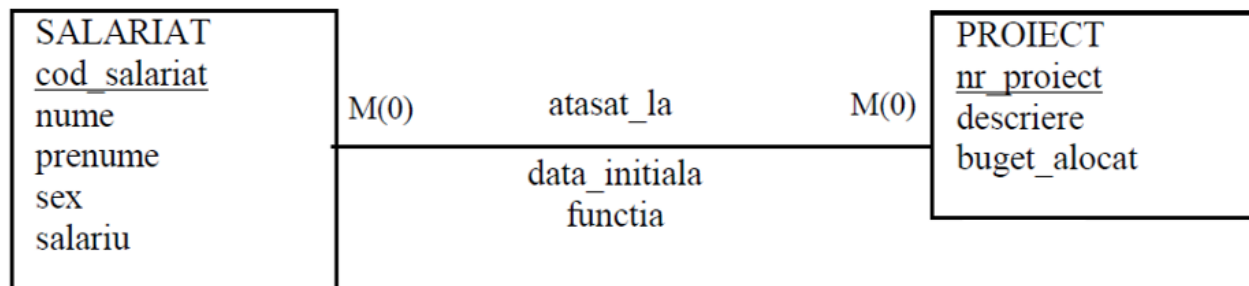
DIAGrame ENTITATE-RELAȚIE

➤ Relație sau atribut?

- Dacă un atribut al unei entități reprezintă cheia primară a unei alte entități, atunci el referă o relație (*cod_departament* în tabelul SALARIAT).

➤ Entitate sau relație?

- Se cercetează cheia primară. Dacă aceasta combină cheile primare a două entități, atunci este vorba de o relație.
- **EX:** cheia primară a relației *atasat_la* combină *cod_salariat* cu *nr_proiect*, prin urmare, **SALARIAT_atasat_la_PROIECT** va defini o relație și nu o entitate.



DIAGrame ENTITATE-RELATIE

Cazuri speciale de entități, relații, atribute și modul lor de reprezentare în cadrul diagramei entitate-relație:

- Dependența (entitatea dependentă)
- Relații recursive
- Moștenirea atributelor (subentitate / superentitate)
- Generalizare, specializare
- Relații de tip 2, 3
- Excluderea relațiilor indirecte
- Eliminarea atributelor derivabile
- Relație sau atribut?
- Entitate sau relație?
- **Reassignarea atributelor indirecte**
- Atribute optionale

DIAGrame ENTITATE-RELATIE

- **Un atribut indirect** nu descrie real relația sau entitatea
- Un atribut indirect este un caz special de relație indirectă care trebuie eliminată
- Prin urmare, attributele indirecte trebuie reasignate deoarece introduc redundanță în date

EX: numărul clădirii în care lucrează un salariat este un atribut al entității DEPARTAMENT și nu este o caracteristică a entității SALARIAT

DIAGrame ENTITATE-RELATIE

Cazuri speciale de entități, relații, atribute și modul lor de reprezentare în cadrul diagramei entitate-relație:

- Dependența (entitatea dependentă)
- Relații recursive
- Moștenirea atributelor (subentitate / superentitate)
- Generalizare, specializare
- Relații de tip 2, 3
- Excluderea relațiilor indirecte
- Eliminarea atributelor derivabile
- Relație sau atribut?
- Entitate sau relație?
- Reassignarea atributelor indirecte
- **Atribute opționale**

DIAGrame ENTITATE-RELATIE

- Există **attribute opționale**, a căror valoare este **uneori necunoscută** (ex: nu avem informația la momentul respectiv), **alteori neaplicabilă** (ex: nu se aplică pentru toate entitățile)
- Aceste attribute trebuie introduse la subentități

Exemplu – Sistem de gestiune a angajaților

În cadrul exemplului avem o superentitate **ANGAJAT**, cu două subentități – **PROGRAMATOR** și **MANAGER**:

- Un **atribut opțional** poate fi → **Tehnologie/Limbaj**
 - Aplicabil pentru **Programatori** (ex: Java, Python, C#)
 - **Neaplicabil** pentru **Manageri**
- Un alt **atribut opțional** poate fi → **Buget gestionat**
 - Aplicabil pentru **Manageri** (ex: 100.000 EUR)
 - **Neaplicabil** pentru **Programatori**

DIAGrame ENTITATE-RELATIE

Algoritmul pentru proiectarea Diagramei E/R

1. identificarea **entităților** din cadrul sistemului analizat;
2. identificarea **relațiilor** (asocierilor) dintre entități și stabilirea **cardinalității**;
3. identificarea **atributelor** aferente entităților și asocierilor dintre entități;
4. stabilirea atributelor de identificare a entităților, adică stabilirea **cheilor primare**;

DIAGrame ENTITATE-RELAȚIE

Aceeași realitate poate fi percepută diferit de către diverși analiști pentru un același sistem => **modele structurale distincte**.

- Construirea **diagramei conceptuale**
- Obținerea **schemelor relaționale**
- **Normalizarea** acestora



=> un **model relațional** care va elimina anumite clase de anomalii ce pot să apară în proiectarea modelului de date

MODELUL E/R EXTINS

- Conceptele de bază ale modelării E/R nu sunt suficiente pentru a reprezenta cerințe complexe
- Modelul E/R susținut cu concepte semantice adiționale definește **modelul E/R extins (EER)**.
 - include toate **conceptele modelului original**
 - + conceptele adiționale de **subclasă, superclasă, moștenire, specializare, generalizare**.

Modelul EER (modelul E/R extins) = Diagrama E/R + concepte adiționale (subclasă, superclasă, moștenire, specializare, generalizare).

DEFICIENȚE ALE MODELULUI E/R

- Cauza: interpretare eronată a sensului unei relații => **capcane de conectare**
- Posibil să necesite restructurarea modelului

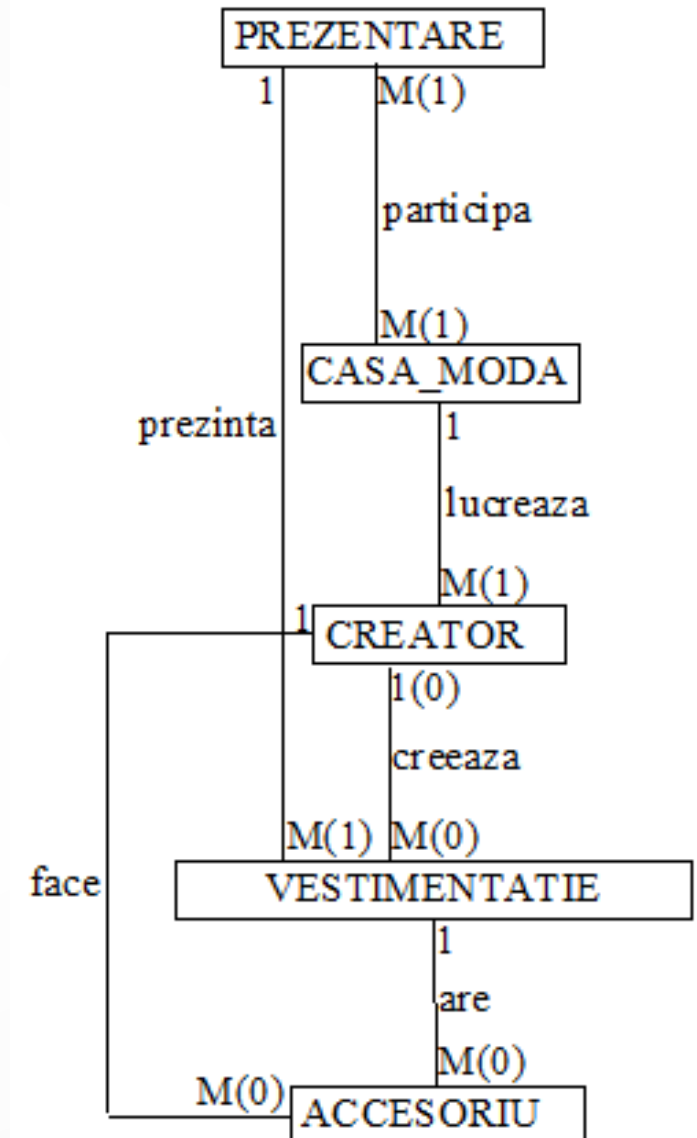
Există 2 clase de capcane de conectare:

- **de întrerupere**
- **în evantai**

DEFICIENȚE ALE MODELULUI E/R

1. Capcane de întrerupere

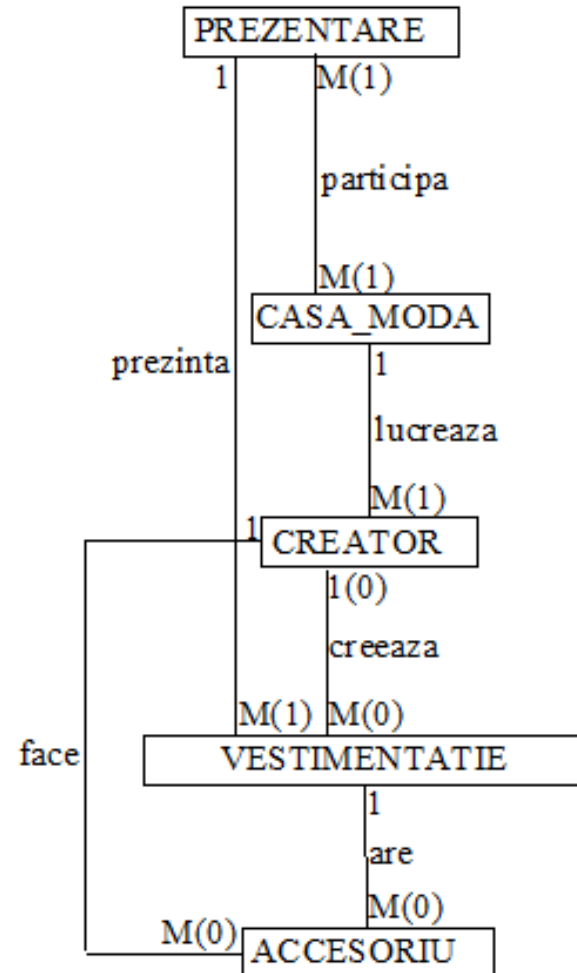
- Modelul sugerează existența unei relații între entități, dar nu există o cale între anumite apariții ale entităților
- Această capcană poate să apară acolo unde există o relație cu **participare parțială** (0 la cardinalitatea minimă) care face parte din calea dintre entitățile ce sunt legate



DEFICIENȚE ALE MODELULUI E/R

- Aceste capcane generează situațiile în care, așa cum a fost proiectat modelul de date, **el nu poate să răspundă la anumite interogări.**

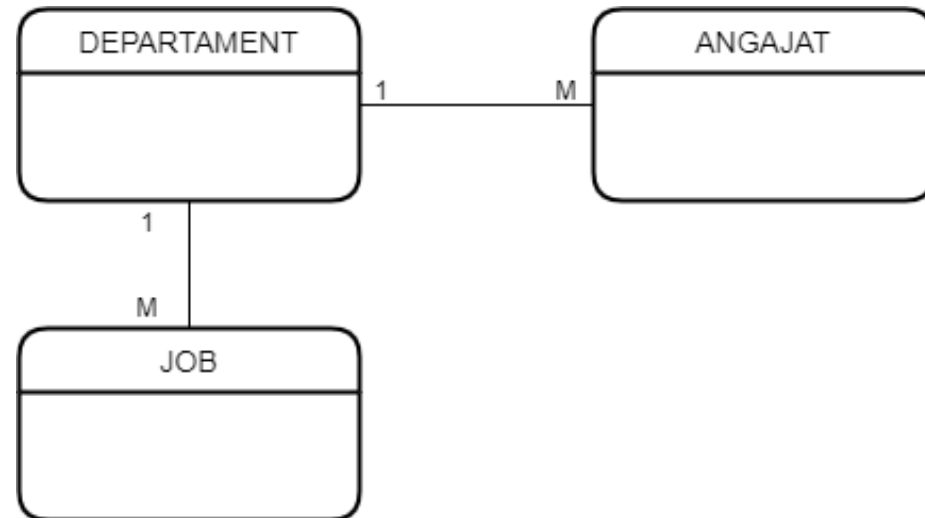
De exemplu: **pentru a afla pentru ce prezentare de modă a fost creată o anumită vestimentație**, a fost necesară introducerea unei legături între entitățile PREZENTARE și VESTIMENTATIE, care însă a generat redundanță în modelul de date



DEFICIENȚE ALE MODELULUI E/R

2. Capcane în evantai

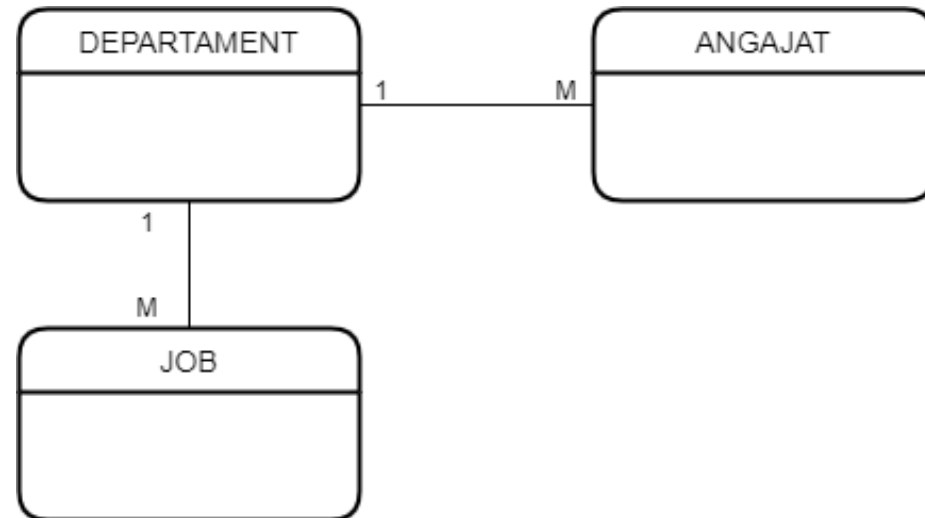
- Modelul ia în considerare o relație între entități, dar calea dintre anumite apariții ale entităților este ambiguă
- Aceste capcane apar când **două sau mai multe relații *one_to_many* provin din aceeași entitate**



DEFICIENȚE ALE MODELULUI E/R

2. Capcane în evantai

- Modelul ia în considerare o relație între entități, dar calea dintre anumite apariții ale entităților este ambiguă
- Aceste capcane apar când **două sau mai multe relații *one_to_many* provin din aceeași entitate**

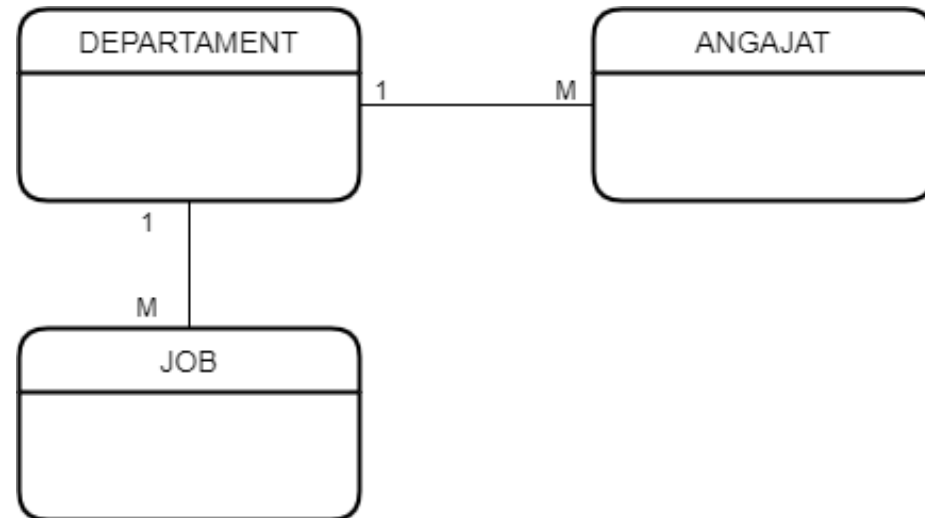


**La ce întrebare nu
răspunde modelul?**

DEFICIENȚE ALE MODELULUI E/R

2. Capcane în evantai

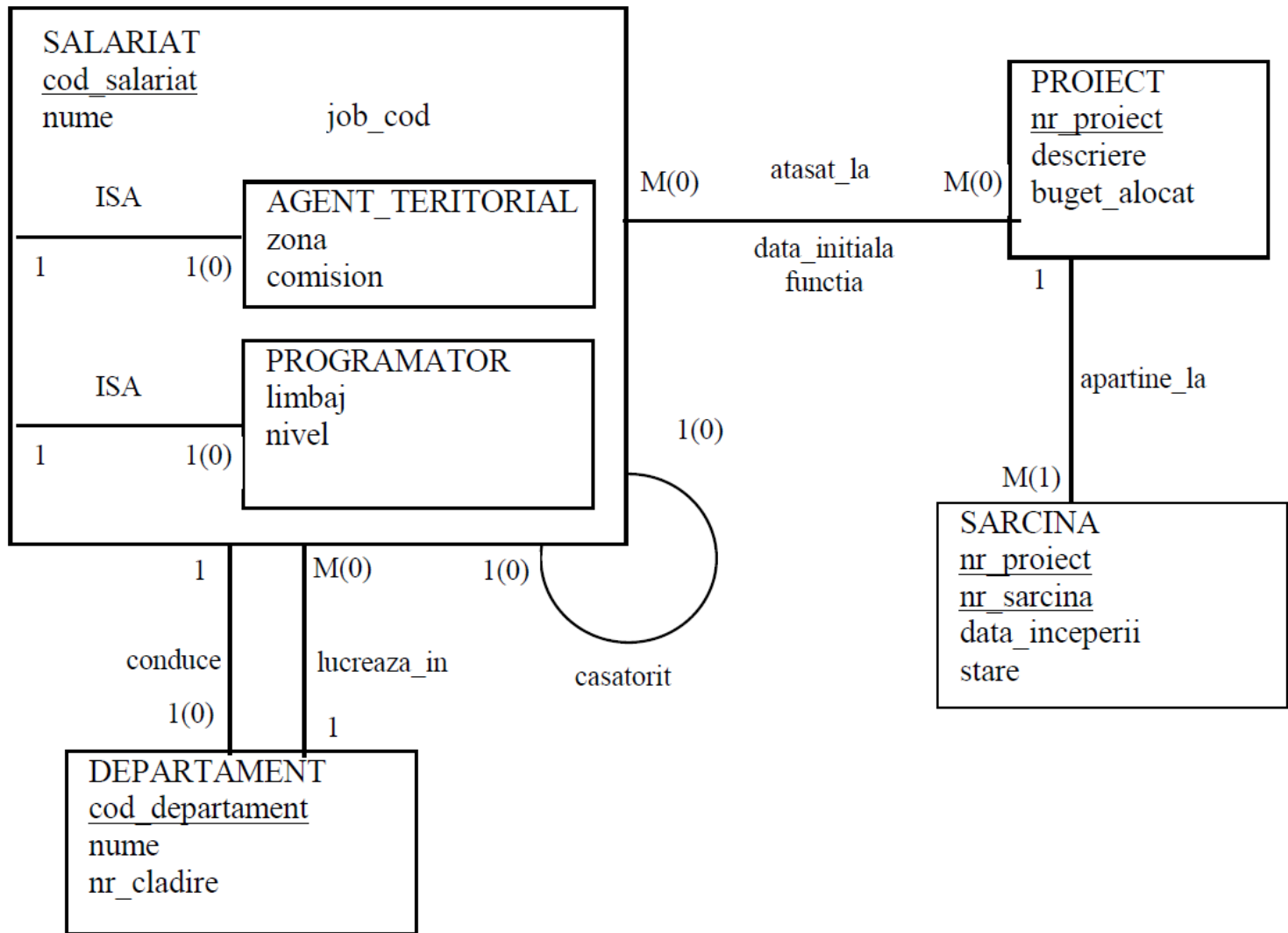
- Modelul ia în considerare o relație între entități, dar calea dintre anumite apariții ale entităților este ambiguă
- Aceste capcane apar când două sau mai multe relații *one_to_many* provin din aceeași entitate



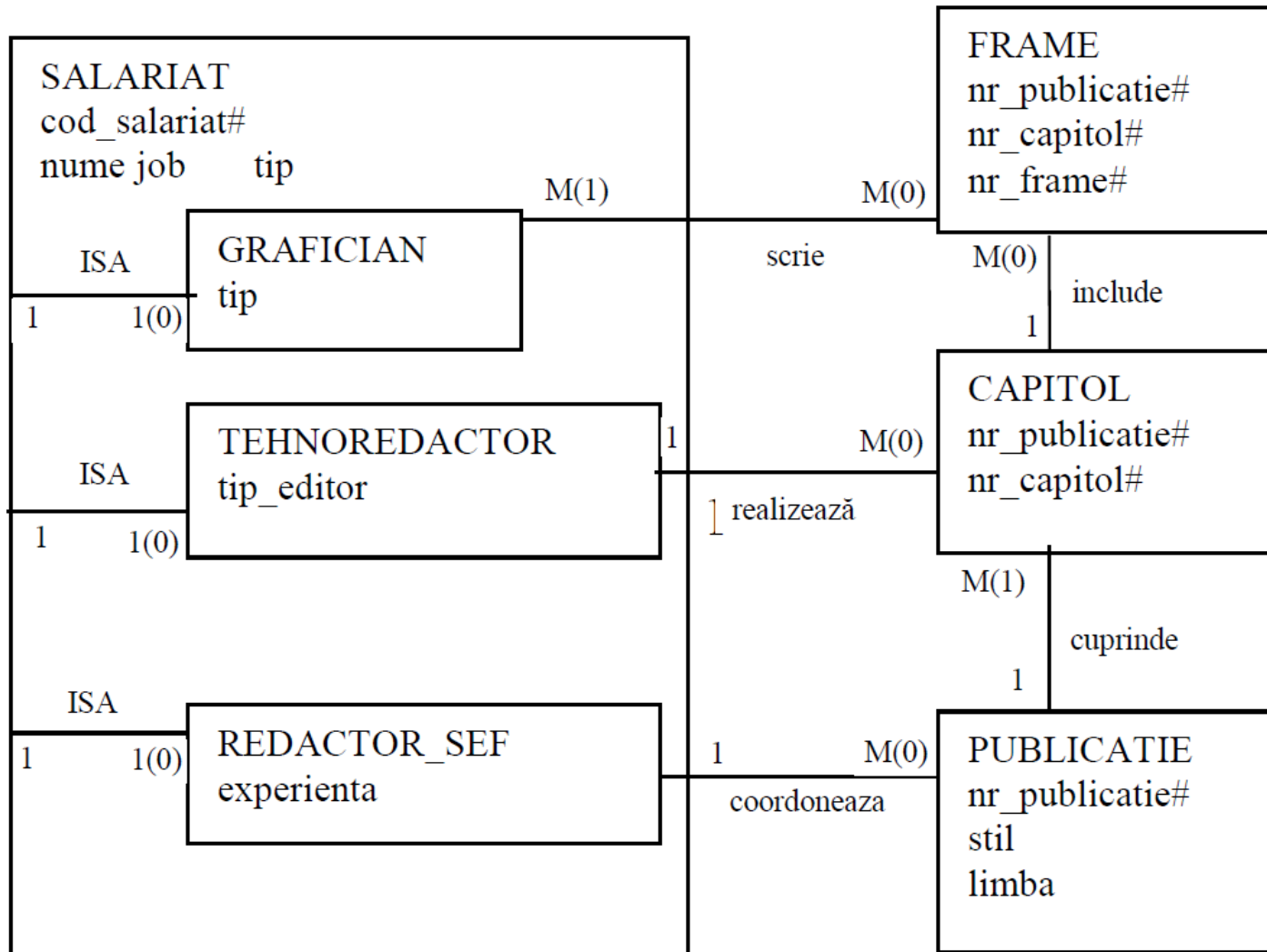
La ce întrebare nu răspunde modelul?

Nu se poate răspunde la întrebarea – **Ce job are un angajat?**

EXEMPLUL 1: Gestiunea activităților din cadrul unei companii



EXEMPLUL 2: Gestiunea activităților de editare dintr-o editură



**Ce fel de entități sunt entitățile: CAPITOL și FRAME?
Cum se implementează?**

TEMĂ

Studiați noțiunile din acest curs și analizați toate exemplele din cadrul acestuia!